Java Logic Building Questions & Practice Guide

165 Structured Problems

From absolute basics to interview-level challenges

Module	Topic	Focus	Week
1	**Easy (1-25)**	Loops, strings, arrays	**Week 1**
2	**Medium (26-50)**	Sorting, searching, core math	**Week 2**
3	**Patterns (51-80)**	Star, number & alphabet patterns	**Week 3**
4	**Sequences (81-90)**	Recurrences: Fibonacci to Recaman	**Week 3**
5	**Number Theory (91-105)**	Divisors, totients, digit logic	**Week 4**
6	**Strings (106-120)**	Stacks, sliding window, DP	**Week 4**
7	**Arrays & Matrix (121-135)**	LIS, BFS/DFS, monotonic stack	**Week 5**
8	**Recursion (136-145)**	Backtracking & pruning	**Week 5**
9	**Hashing & Sim (146-155)**	HashMap, BFS, simulation	**Week 6**

Each problem includes the core concepts it tests and a solving hint.

The Week column above lays out a six-week study plan; a complexity reference closes the guide.

How to Use This Book

This guide is organised strictly by increasing difficulty, so you can work top to bottom and always be appropriately challenged. Each numbered problem follows the same layout:

Element	What it means
Bold statement	The problem to solve, with a worked example where helpful.
◆ Concepts	The data structures or techniques the problem is designed to teach.
■ Hint	A nudge toward the intended approach — read only if you are stuck.
[Badge]	The topic category, for quick scanning and revision.

A Reliable Problem-Solving Method

Understand. Read the problem twice. Note what is given, what is asked, and the constraints.

Example. Trace 2-3 inputs by hand, including edge cases: empty, single element, negatives, maximum size.

Brute force. Write the simplest correct solution first. Correctness before cleverness.

Analyse. State the time and space complexity of what you just wrote.

Optimise. Look for a known pattern: sorting, hashing, two pointers, sliding window, or dynamic programming.

Verify. Re-run your hand-traced examples and edge cases against the final code.

Getting the Most From This Guide

- Give every problem a real attempt (20-30 minutes) before reading its hint.
- Time yourself once you are comfortable: about 15 minutes for easy, 30-45 for medium and hard.
- After every ten problems, pause and name the patterns that keep recurring.

Module 1 — Easy (Problems 1–25)

These problems build the core instincts every programmer needs: loops, conditionals, and simple data manipulation. Aim to solve each in well under ten minutes.

Numbers & Math

1. Check whether a number is **even or odd**. [Numbers]
   - ◆ *Concepts: Modulo operator, conditionals*
   - ■ Hint: n % 2 == 0 means even.
2. Find the **largest of three numbers** without using `Math.max()`. [Numbers]
   - ◆ *Concepts: Comparison logic*
   - ■ Hint: Chain if/else comparisons, or nest ternary operators.
3. Print the **sum of digits** of a number — e.g. 123 → 6. [Numbers]
   - ◆ *Concepts: Digit extraction, loops*
   - ■ Hint: Repeatedly take n%10 and then n/=10 until n is 0.
4. Check if a number is a **palindrome** — e.g. 121 → true. [Numbers]
   - ◆ *Concepts: Number reversal*
   - ■ Hint: Reverse the number and compare with the original.
5. Find the **factorial** of a number using a loop. [Numbers]
   - ◆ *Concepts: Iteration, overflow awareness*
   - ■ Hint: Use long; factorials overflow int quickly (13! > int max).
6. Print all **divisors** of a number. [Numbers]
   - ◆ *Concepts: Loops, optimisation*
   - ■ Hint: Only loop to sqrt(n); add both i and n/i for each divisor found.
7. Check if a number is **prime**. [Numbers]
   - ◆ *Concepts: Primality testing*
   - ■ Hint: Test divisibility only up to sqrt(n); handle n < 2 as not prime.
8. Find the **GCD** of two numbers. [Numbers]
   - ◆ *Concepts: Euclidean algorithm*
   - ■ Hint: gcd(a,b) = gcd(b, a%b); base case gcd(a,0) = a.
9. Reverse a given integer — e.g. 1234 → 4321. [Numbers]
   - ◆ *Concepts: Digit manipulation*
   - ■ Hint: Build the result as result = result*10 + n%10.
10. Check if a number is an **Armstrong number** — e.g. 153 = $1^3+5^3+3^3$. [Numbers]
    - ◆ *Concepts: Digit powers*
    - ■ Hint: Count digits d, then sum each digit raised to the power d.

Strings

11. Reverse a string **without** using `StringBuilder`. [Strings]

◆ *Concepts: Char arrays, two pointers*

■ Hint: Swap characters from both ends moving inward.

12. Check if a string is a **palindrome**. [Strings]

◆ *Concepts: Two pointers*

■ Hint: Compare characters at i and length-1-i.

13. Count **vowels and consonants** in a string. [Strings]

◆ *Concepts: Character classification*

■ Hint: Normalise case first; check membership in "aeiou".

14. Remove all **spaces** from a string. [Strings]

◆ *Concepts: String traversal*

■ Hint: Build a new string skipping whitespace characters.

15. Count the **frequency of each character**. [Strings]

◆ *Concepts: Hashing / int[26]*

■ Hint: For lowercase letters, an int[26] array is faster than a HashMap.

16. Check whether two strings are **anagrams**. [Strings]

◆ *Concepts: Frequency counting, sorting*

■ Hint: Equal length + identical character counts, or compare sorted forms.

17. Convert the **first letter of each word** to uppercase (title case). [Strings]

◆ *Concepts: Word boundaries*

■ Hint: Capitalise a letter when the previous character was a space.

18. Find the **longest word** in a sentence. [Strings]

◆ *Concepts: Tokenisation*

■ Hint: Split on whitespace; track the longest token seen.

Arrays

19. Find the **maximum and minimum** in an array. [Arrays]

◆ *Concepts: Single-pass scan*

■ Hint: Track both in one loop; initialise from arr[0], not 0.

20. Calculate the **average** of all elements. [Arrays]

◆ *Concepts: Accumulation*

■ Hint: Use a long sum to avoid overflow, then cast for division.

21. Count **even and odd** numbers. [Arrays]

◆ *Concepts: Modulo, counters*

■ Hint: One pass with two counters.

22. **Reverse** an array in place. [Arrays]

◆ *Concepts: Two pointers, swapping*

■ Hint: Swap arr[i] and arr[n-1-i] up to the midpoint.

23. Check if an array contains a **duplicate**. [Arrays]

◆ *Concepts: HashSet*

■ Hint: Add to a set; a failed add signals a duplicate. O(n).

24. Find the **second largest** element. [Arrays]

◆ *Concepts: Single-pass tracking*

■ Hint: Track largest and secondLargest together; mind duplicates.

25. **Merge two sorted arrays** into one sorted array. [Arrays]

◆ *Concepts: Merge step of merge sort*

■ Hint: Two pointers advancing through both arrays.

Module 1 Tips

- Always initialise max/min from the first element, never from 0 — negatives will break it.
- For lowercase-only character counts, prefer an int[26] array over a HashMap.
- Watch for integer overflow in factorial and sum problems: reach for long.
- Handle empty and single-element inputs explicitly — they are the most common bug source.

Module 2 — Medium (Problems 26–50)

Medium problems reward thinking about algorithms and complexity rather than brute force. Give each 20-30 minutes before consulting the hint.

Number Logic

26. Print all primes from 1 to N using the **Sieve of Eratosthenes**. [Numbers]

◆ *Concepts: Sieve, boolean array*

■ Hint: Mark multiples of each prime as composite; start marking at p*p.

27. Print the **Fibonacci series** to N terms, both iteratively and recursively. [Numbers]

◆ *Concepts: Recursion, iteration*

■ Hint: Naive recursion is O(2^n); the loop is O(n).

28. Find all **perfect numbers** from 1 to 1000 — e.g. 6 = 1+2+3. [Numbers]

◆ *Concepts: Divisor sums*

■ Hint: A perfect number equals the sum of its proper divisors.

29. Check if a number is a **happy number**. [Numbers]

◆ *Concepts: Cycle detection*

■ Hint: Repeatedly sum squares of digits; use a set (or Floyd) to detect loops.

30. Find the **LCM** of two numbers without the GCD shortcut. [Numbers]

◆ *Concepts: Multiples*

■ Hint: Increment the larger number until it is divisible by the smaller.

31. Print **Pascal's Triangle** to N rows. [Numbers]

◆ *Concepts: Combinatorics, DP*

■ Hint: Each entry is the sum of the two entries directly above it.

32. Print all numbers up to a limit whose **digit sum equals N**. [Numbers]

◆ *Concepts: Digit processing*

■ Hint: Reuse your digit-sum routine from problem 3.

33. Count the **trailing zeroes in N!**. [Numbers]

◆ *Concepts: Prime factorisation*

■ Hint: Count factors of 5: floor(N/5)+floor(N/25)+floor(N/125)+...

String Logic

34. Generate all **permutations** of a string. [Strings]

◆ *Concepts: Backtracking*

■ Hint: Swap-based recursion, or fix one character and permute the rest.

35. **Compress** a string with run-length encoding — `aaabbc` → `a3b2c1`. [Strings]

◆ *Concepts: Counting runs*

■ Hint: Single pass with a counter; flush on character change.

36. Check if one string is a **rotation** of another — `abcd`, `cdab`. [Strings]

◆ *Concepts: String concatenation trick*

■ Hint: b is a rotation of a iff b is a substring of a+a and lengths match.

37. Find the **first non-repeating character**. [Strings]

◆ *Concepts: Frequency map, ordering*

■ Hint: Count in one pass, then scan left to right for the first count of 1.

38. Count the **words in a sentence**, handling multiple spaces. [Strings]

◆ *Concepts: Tokenisation*

■ Hint: Split on one-or-more whitespace, or count word-start transitions.

39. **Reverse each word** individually, keeping word order. [Strings]

◆ *Concepts: Split and rebuild*

■ Hint: Reverse each token, then rejoin with single spaces.

40. Find the **longest palindromic substring**. [Strings]

◆ *Concepts: Expand around center*

■ Hint: Try each of the 2n-1 centers; expand outward while characters match.

Arrays & Logic

41. Find the **missing number** in an array of 1 to N. [Arrays]

◆ *Concepts: Math / XOR*

■ Hint: Expected sum n(n+1)/2 minus the actual sum, or XOR all values with 1..n.

42. Move all **zeroes to the end**, keeping order of non-zeroes. [Arrays]

◆ *Concepts: Two pointers*

■ Hint: Write index advances only for non-zero elements.

43. Find the subarray with the **maximum sum** (Kadane's algorithm). [Arrays]

◆ *Concepts: Dynamic programming*

■ Hint: Reset the running sum to 0 whenever it turns negative.

44. **Rotate an array** right by K positions. [Arrays]

◆ *Concepts: Reversal trick*

■ Hint: Reverse whole array, reverse first K, reverse the rest. O(1) space.

45. Find all **pairs summing to a target**. [Arrays]

◆ *Concepts: Hashing*

■ Hint: Store seen values in a set; check for target-current.

46. Print a 2D matrix in **spiral order**. [Arrays]

◆ *Concepts: Boundary pointers*

■ Hint: Maintain top, bottom, left, right; shrink after each side.

47. Validate a **Sudoku board**. [Arrays]

◆ *Concepts: Hashing, constraints*

■ Hint: Check rows, columns, and 3x3 boxes for duplicate digits.

48. Sort an array of 0s, 1s and 2s in one pass (**Dutch National Flag**). [Arrays]

◆ *Concepts: Three pointers*

■ Hint: Pointers low, mid, high partition the array as mid advances.

49. Find the **longest consecutive sequence** in an unsorted array. [Arrays]

◆ *Concepts: HashSet*

■ Hint: Only start counting from a number whose predecessor is absent. O(n).

50. Find two numbers with the **maximum product**. [Arrays]

◆ *Concepts: Extremes tracking*

■ Hint: Track two largest and two smallest — two negatives can win.

Module 2 Tips

- Kadane's algorithm (43): keep a running sum and a best sum; reset the running sum at zero.
- The rotation reversal trick (44) gives O(1) space — far better than an extra array.
- Dutch National Flag (48) uses three pointers and sorts in a single pass.
- Trailing zeroes in N! (33) come only from factors of 5, since 2s are always more plentiful.
- For maximum product (50), remember two negatives multiply to a positive.

Module 3 — Patterns (Problems 51–80)

Pattern printing is the fastest way to master nested loops. For each pattern, work out the relationship between the row number, the number of leading spaces, and the characters printed. Use an outer loop for rows and inner loops for spaces and content.

Read This First

- Outer loop = rows (i from 1 to N). Inner loops handle spaces, then characters.
- Leading spaces for a centered pyramid are usually (N - i).
- For hollow shapes, print a character only on a border: first/last row or first/last column.
- Track a separate counter for number and alphabet patterns.

Star Patterns (51–60)

51. Right-Angled Triangle

```
*
* *
* * *
* * * *
* * * * *
```

52. Inverted Right Triangle

```
* * * * *
* * * *
* * *
* *
*
```

53. Right-Aligned Triangle

```
        *
      * *
    * * *
  * * * *
* * * * *
```

54. Centered Pyramid

```
    *
   * *
  * * *
 * * * *
* * * * *
```

55. Inverted Pyramid

```
* * * * *
 * * * *
  * * *
   * *
    *
```

56. Diamond

```
    *
   * *
  * * *
 * * * *
* * * * *
 * * * *
  * * *
   * *
    *
```

57. Hollow Square

```
* * * * *
*       *
*       *
*       *
* * * * *
```

58. Hollow Triangle

```
*
* *
*   *
*     *
* * * * *
```

59. Butterfly

```
*       *
* *   * *
* * * * *
* *   * *
*       *
```

60. X Shape

```
* * * * *
 *     *
  *   *
   * *
    *
   * *
  *   *
 *     *
* * * * *
```

Number Patterns (61–70)

61. Number Triangle

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

62. Reverse Number Triangle

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

63. Same Number per Row

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

64. Floyd's Triangle

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

65. Pascal's Triangle

```
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

66. Number Pyramid

```
    1
   2 3
  4 5 6
 7 8 9 10
```

67. Palindrome Triangle

```
    1
   121
  12321
 1234321
123454321
```

68. Descending Triangle

```
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```

69. Multiplication Grid

```
1  2  3  4  5
2  4  6  8  10
3  6  9  12 15
4  8  12 16 20
5  10 15 20 25
```

70. Zigzag 0/1 Pattern

```
1 0 1 0 1
0 1 0 1 0
1 0 1 0 1
0 1 0 1 0
```

Alphabet Patterns (71–80)

71. Alphabet Triangle

```
A
A B
A B C
A B C D
A B C D E
```

72. Reverse Alphabet Triangle

```
A B C D E
A B C D
A B C
A B
A
```

73. Same Letter per Row

```
A
B B
C C C
D D D D
E E E E E
```

74. Alphabet Pyramid

```
    A
   A B
  A B C
 A B C D
A B C D E
```

75. Reverse Alphabet Pyramid

```
A B C D E
 B C D E
  C D E
   D E
    E
```

76. K-Shape Alphabet

```
E
D
C
B
A
B
C
D
E
```

77. Diagonal Alphabet

```
A
  B
    C
      D
        E
```

78. Mirrored Triangle

```
A         A
A B     B A
A B C C B A
A B C D C B A
```

79. Hollow Alphabet Diamond

```
    A
   A B
  A   C
   A B
    A
```

80. Spiral Alphabet (concept)

```
A B C D E
        F
K J I H G
L
M N O P Q
```

Pattern Tips

- Test every pattern with N=5 first, then check the N=1 edge case.
- For pyramids, spaces and characters together should form a constant width per row.
- Floyd's triangle (64) needs a running counter that never resets between rows.
- Palindrome triangles (67) combine descending spaces, an ascending run, and a descending run.

Module 4 — Sequences (Problems 81–90)

Sequence problems train you to spot mathematical structure and translate it into iterative or recursive code. For each one, derive the recurrence relation or closed-form on paper before you write a single line.

81. Arithmetic Sequence

Given first term a, common difference d, and count N, print all N terms and their sum. Then compute the 50th term directly, without iterating.

◆ *Concepts: Closed-form, series sums*

■ Hint: T(n) = a + (n-1)d; Sum = n/2 × (2a + (n-1)d).

82. Geometric Sequence

Given first term a and common ratio r, print N terms and their sum. Handle r = 0 and r = 1 as special cases.

◆ *Concepts: Exponential growth, edge cases*

■ Hint: T(n) = a × r^(n-1); Sum = a(r^n - 1)/(r - 1) for r ≠ 1.

83. Fibonacci — Four Ways

Implement Fibonacci four ways: iterative, plain recursion, memoised recursion, and matrix exponentiation. Compare their running time for N = 50.

◆ *Concepts: Recursion, memoisation, fast exponentiation*

■ Hint: F(n) = F(n-1) + F(n-2). Naive recursion is O(2^n); matrix power is O(log n).

84. Triangular Numbers

Print the first N triangular numbers, test whether a given value is triangular, and find the smallest triangular number greater than K.

◆ *Concepts: Series, inverse formula*

■ Hint: T(n) = n(n+1)/2. A value x is triangular iff 8x+1 is a perfect square.

85. Squares, Cubes, and Both

Print perfect squares up to N^2 and cubes up to N^3. Then list every number up to 1000 that is simultaneously a perfect square and a perfect cube.

◆ *Concepts: Perfect powers*

■ Hint: Numbers that are both are perfect sixth powers: 1, 64, 729, ...

86. Collatz Sequence

For a positive integer N, repeatedly apply: if even, halve it; if odd, use 3N+1. Print the chain until it reaches 1 and count the steps. Then find which starting value up to 100 gives the longest chain.

◆ *Concepts: Simulation, iteration*

■ Hint: Use long for intermediate values — 3N+1 can grow large before collapsing.

87. Padovan Sequence

Generate the first N terms where P(n) = P(n-2) + P(n-3), with P(0)=P(1)=P(2)=1.

◆ *Concepts: Multi-term recurrence*

■ Hint: Sequence: 1, 1, 1, 2, 2, 3, 4, 5, 7, 9, 12, 16, 21, ...

88. Recaman's Sequence

Generate N terms: start at 0; at step k, subtract k if the result is positive and unused, otherwise add k.

◆ *Concepts: HashSet, conditional logic*

■ Hint: Track seen values in a set to decide subtract-versus-add at each step.

89. Look-and-Say Sequence

Starting from "1", generate N terms by reading each term aloud: count runs of identical digits.

◆ *Concepts: String building, run counting*

■ Hint: 1 → 11 → 21 → 1211 → 111221. Terms are Strings — they outgrow numeric types.

90. Spiral Number Matrix

Fill an N×N matrix with 1..N² in clockwise spiral order from the top-left, print it, then report the row and column of a given value K.

◆ *Concepts: Boundary pointers, 2D arrays*

■ Hint: Maintain top, bottom, left, right boundaries; shrink each after filling a side.

Sequence Tips

- Always derive the recurrence or closed-form before coding — it prevents off-by-one errors.
- Memoise recursive sequences with a HashMap or array to avoid exponential recomputation.
- Collatz (86) and Recaman (88) are best done iteratively to avoid deep recursion.
- Look-and-Say (89) terms grow fast — keep them as Strings, never as int or long.
- Test each with N = 0, N = 1, and a large N to catch boundary bugs.

Module 5 — Number Theory (Problems 91–105)

These problems reward knowing the mathematical definition first, then choosing an efficient implementation. Many have elegant shortcuts worth learning.

91. Check if a number is a **Kaprekar number** — e.g. $45^2 = 2025$, and 20+25 = 45. [Number Theory]

◆ *Concepts: Squaring, digit splitting*

■ Hint: Split the square at the digit count of the original and sum the parts.

92. List all **automorphic numbers** up to N — e.g. $25^2 = 625$ ends in 25. [Number Theory]

◆ *Concepts: String suffix check*

■ Hint: Check whether the square, as a string, ends with the original number.

93. Check if a number is a **Neon number** — e.g. $9 \rightarrow 9^2 = 81 \rightarrow 8+1 = 9$. [Number Theory]

◆ *Concepts: Digit sum of a square*

■ Hint: The digit sum of the square must equal the original number.

94. List all **abundant numbers** up to N (proper divisor sum exceeds the number). [Number Theory]

◆ *Concepts: Divisor sums*

■ Hint: Sum proper divisors, then compare with the number itself.

95. Compute the **Nth Catalan number**. [Number Theory]

◆ *Concepts: Combinatorics, DP*

■ Hint: C(n) = C(n-1) × 2(2n-1) / (n+1). Counts valid bracketings and BSTs.

96. Check if a number is a **Spy number** (digit sum equals digit product). [Number Theory]

◆ *Concepts: Digit aggregation*

■ Hint: Maintain a running sum and product, then compare.

97. Find the **digital root** (repeatedly sum digits to a single digit). [Number Theory]

◆ *Concepts: Digit sums, modular shortcut*

■ Hint: For n > 0, digital root = 1 + (n-1) % 9.

98. Check if a number is a **Harshad (Niven) number** — divisible by its digit sum. [Number Theory]

◆ *Concepts: Divisibility*

■ Hint: Compute the digit sum, then test n % digitSum == 0.

99. Generate all **ugly numbers** up to N (only prime factors 2, 3, 5). [Number Theory]

◆ *Concepts: Dynamic programming, pointers*

■ Hint: Three pointers i2, i3, i5; each step take the minimum candidate.

100. Find the **Nth Fibonacci** via matrix exponentiation in O(log N). [Number Theory]

◆ *Concepts: Fast exponentiation, matrices*

■ Hint: Raise [[1,1],[1,0]] to the Nth power with fast matrix power.

101. Find the **smallest number with exactly N divisors**. [Number Theory]

◆ *Concepts: Divisor counting*

■ Hint: Scan upward, counting divisors up to sqrt, until you hit exactly N.

102. Check if a number is a **Disarium number** — e.g. 135 = $1^1+3^2+5^3$. [Number Theory]

◆ *Concepts: Positional digit powers*

■ Hint: Raise each digit to the power of its position from the left.

103. List all **emirp primes** up to N (prime whose reverse is a different prime). [Number Theory]

◆ *Concepts: Primality, reversal*

■ Hint: Both n and its reverse must be prime and must differ.

104. Determine if a number is a **sum of two squares**. [Number Theory]

◆ *Concepts: Perfect-square testing*

■ Hint: For i from 0 to sqrt(n), test whether n - i^2 is a perfect square.

105. Compute **Euler's totient** ϕ(N) — count of integers in 1..N coprime with N. [Number Theory]

◆ *Concepts: Prime factorisation*

■ Hint: For each prime factor p of N, multiply the result by (1 - 1/p).

Number Theory Tips

- Ugly numbers (99): the three-pointer method is O(N), far better than trial division.
- Euler's totient (105): iterate only over distinct prime factors, not all numbers up to N.
- The digital-root shortcut (97) removes the loop entirely — a neat modular-arithmetic result.
- Matrix exponentiation (100) is the standard way to reach the Nth Fibonacci in logarithmic time.

Module 6 — Advanced Strings (Problems 106–120)

106. Check for **balanced brackets** — `{[()]}` valid, `{[(])}` invalid. [Strings]

◆ *Concepts: Stack*

■ Hint: Push openers; on a closer, pop and confirm the pair matches.

107. Find the **longest common prefix** across an array of strings. [Strings]

◆ *Concepts: String comparison*

■ Hint: Sort, then compare only the first and last strings character by character.

108. Find the **minimum bracket reversals** needed to balance a bracket string. [Strings]

◆ *Concepts: Stack, arithmetic*

■ Hint: After cancelling matched pairs, combine the leftover open and close counts.

109. Count the number of **palindromic substrings**. [Strings]

◆ *Concepts: Expand around center*

■ Hint: Expand from each of the 2n-1 centers while characters match.

110. Implement your own `indexOf()`. [Strings]

◆ *Concepts: Pattern matching*

■ Hint: Compare the pattern at each start position; KMP if you want O(n+m).

111. **Reverse only the vowels** in a string, leaving other letters in place. [Strings]

◆ *Concepts: Two pointers*

■ Hint: Advance both pointers to vowels, then swap and step inward.

112. Find the **longest substring without repeating characters**. [Strings]

◆ *Concepts: Sliding window, hashing*

■ Hint: Move the left edge past the last occurrence of a repeated character.

113. Match a string against a **wildcard pattern** (* any run, ? any single char). [Strings]

◆ *Concepts: Dynamic programming*

■ Hint: dp[i][j] = does s[..i] match p[..j]; handle * as zero-or-more.

114. Find the **shortest common supersequence** of two strings. [Strings]

◆ *Concepts: LCS, dynamic programming*

■ Hint: Length = len(a) + len(b) - LCS(a, b).

115. Count the ways to **decode a numeric string** (1=A ... 26=Z). [Strings]

◆ *Concepts: Dynamic programming*

■ Hint: dp[i] = dp[i-1] (valid single) + dp[i-2] (valid pair 10-26).

116. Find **duplicate words** in a paragraph with their counts. [Strings]

◆ *Concepts: Hashing, ordering*

■ Hint: A LinkedHashMap preserves first-seen order while counting.

117. Convert between **Roman numerals and integers**, both directions. [Strings]

◆ *Concepts: Mapping, greedy*

■ Hint: Right-to-left: subtract when a symbol is smaller than the one after it.

118. Find the **most frequent word** in a list. [Strings]

◆ *Concepts: Hashing*

■ Hint: Count in a HashMap and track the running maximum in one pass.

119. Check whether B is a **subsequence** of A — e.g. "ace" in "abcde". [Strings]

◆ *Concepts: Two pointers*

■ Hint: Advance B's pointer only on a match; B is a subsequence if it is exhausted.

120. Implement **basic string compression** — `aabccc` → `a2b1c3`. [Strings]

◆ *Concepts: Run counting*

■ Hint: One pass with a counter; append the char and its count on each change.

String Tips

- Balanced brackets (106) is the canonical stack problem — it extends to expression parsing.
- The sliding window (112) with a last-seen map solves the substring problem in O(n).
- Wildcard matching (113): the tricky case is * matching an empty run — handle dp[0][j] carefully.
- Subsequence checking (119) is O(n) with two pointers; never do it in $O(n^2)$.

Module 7 — Arrays & Matrix (Problems 121–135)

121. Find the **equilibrium index** where the left sum equals the right sum. [Arrays / Matrix]

◆ *Concepts: Prefix sums*

■ Hint: Track a left sum; the right sum is total - left - arr[i].

122. Find the **longest increasing subsequence**. [Arrays / Matrix]

◆ *Concepts: DP, binary search*

■ Hint: O(n²) DP, or O(n log n) with a patience-sorting tails array.

123. Perform **binary search on a rotated sorted array**. [Arrays / Matrix]

◆ *Concepts: Modified binary search*

■ Hint: One half is always sorted; decide which, then narrow the range.

124. Find the **minimum jumps** to reach the end of an array. [Arrays / Matrix]

◆ *Concepts: Greedy*

■ Hint: Track the current reach, the farthest reachable, and the jump count.

125. Find a **saddle point** (minimum in its row, maximum in its column). [Arrays / Matrix]

◆ *Concepts: Matrix scanning*

■ Hint: Find each row minimum, then verify it is its column's maximum.

126. Rotate a **matrix 90° clockwise in place**. [Arrays / Matrix]

◆ *Concepts: Transpose and reverse*

■ Hint: Transpose the matrix, then reverse each row.

127. Count the **number of islands** in a binary grid. [Arrays / Matrix]

◆ *Concepts: DFS / BFS, flood fill*

■ Hint: For each unvisited 1, flood-fill its component and increment a counter.

128. Find the **next greater element** for every position. [Arrays / Matrix]

◆ *Concepts: Monotonic stack*

■ Hint: Pop from the stack while the current element exceeds the top.

129. Maximise profit in the **single-transaction stock problem**. [Arrays / Matrix]

◆ *Concepts: Running minimum*

■ Hint: Track the lowest price so far and the best profit against it.

130. Find the **maximum product subarray** (values may be negative). [Arrays / Matrix]

◆ *Concepts: Dynamic programming*

■ Hint: Track both the running max and min — a negative flips them.

131. **Search a sorted 2D matrix** in O(m + n). [Arrays / Matrix]

◆ *Concepts: Staircase search*

■ Hint: Start at the top-right; move left if too big, down if too small.

132. Count **inversions** in an array (pairs iarr[j]). [Arrays / Matrix]

◆ *Concepts: Modified merge sort*

■ Hint: Count cross-inversions during the merge step. O(n log n).

133. Find the **majority element** appearing more than N/2 times. [Arrays / Matrix]

◆ *Concepts: Boyer-Moore voting*

■ Hint: Maintain a candidate and a vote counter; verify with a second pass.

134. Rearrange an array so **positives and negatives alternate**. [Arrays / Matrix]

◆ *Concepts: Partitioning*

■ Hint: Separate the two groups, then interleave; handle unequal counts.

135. Find the **minimum meeting rooms** needed for a set of intervals. [Arrays / Matrix]

◆ *Concepts: Sorting, min-heap*

■ Hint: Sort by start; a min-heap of end times gives the concurrent count.

Arrays & Matrix Tips

- Longest increasing subsequence (122): the O(n log n) version keeps a 'tails' array with binary search.
- Rotated binary search (123): always identify the sorted half before deciding where to look.
- The monotonic stack (128) also solves largest-rectangle-in-histogram and trapping-rain-water.
- Boyer-Moore voting (133) needs a verification pass — a majority is not guaranteed to exist.

Module 8 — Recursion & Backtracking (Problems 136–145)

Backtracking follows one template: choose, explore, then un-choose. Draw the recursion tree for a tiny input before you code, and always define your base case first.

136. Solve the **Tower of Hanoi** for N disks, printing each move. [Recursion]

◆ *Concepts: Recursion*

■ Hint: Move N-1 to the spare peg, move disk N to the target, then move N-1 across.

137. Generate the **power set** (all subsets) of an array. [Recursion]

◆ *Concepts: Include/exclude recursion*

■ Hint: At each index, branch on including or skipping the element. 2^N subsets.

138. Solve the **N-Queens** problem. [Recursion]

◆ *Concepts: Backtracking, constraint sets*

■ Hint: Track used columns and both diagonals for O(1) validity checks.

139. Find all **combinations summing to a target**, elements reusable. [Recursion]

◆ *Concepts: DFS with a start index*

■ Hint: Do not advance the index when reusing the same element.

140. Solve a **Sudoku** puzzle by backtracking. [Recursion]

◆ *Concepts: Constraint propagation*

■ Hint: Try 1-9 in each empty cell; recurse if valid, undo on failure.

141. Generate all **valid parentheses** combinations for N pairs. [Recursion]

◆ *Concepts: Backtracking with counts*

■ Hint: Add '(' while open < N; add ')' while close < open.

142. Find a **path through a maze**, moving only right and down. [Recursion]

◆ *Concepts: Recursion, path marking*

■ Hint: Mark the cell on the way in and unmark on backtrack; count paths too.

143. Print all **permutations of an array**, skipping duplicates. [Recursion]

◆ *Concepts: Backtracking, dedup*

■ Hint: Sort first; skip a value equal to the previous unused one at each level.

144. Solve the **word search** in a 2D character grid. [Recursion]

◆ *Concepts: DFS in four directions*

■ Hint: Mark visited cells during recursion; restore them when returning.

145. Generate all **binary strings of length N without consecutive 1s**. [Recursion]

◆ *Concepts: Recursion with state*

■ Hint: If the last placed bit was 1, the next must be 0; otherwise branch both.

Recursion Tips

- The universal backtracking template is: choose, explore, un-choose.
- N-Queens (138): three boolean sets — column, diagonal, anti-diagonal — make validity O(1).
- Combination Sum (139): sort the input so you can prune once the running sum exceeds the target.
- Always draw the recursion tree for N=3 before coding — it exposes the base case instantly.

Module 9 — Hashing & Simulation (Problems 146–155)

146. Find the **first repeated element** in an array. [Hashing / Simulation]

◆ *Concepts: HashSet*

■ Hint: Add each value; the first failed add is your answer.

147. Group **anagrams** from a list of strings. [Hashing / Simulation]

◆ *Concepts: Hashing with a canonical key*

■ Hint: Use the sorted string (or a count signature) as the map key.

148. Find the **longest subarray summing to K**. [Hashing / Simulation]

◆ *Concepts: Prefix sums, hashing*

■ Hint: Store the first index of each prefix sum; look up prefix - K.

149. Count **distinct elements in every window of size K**. [Hashing / Simulation]

◆ *Concepts: Sliding window, frequency map*

■ Hint: The map's size is the distinct count; drop the outgoing element.

150. Build a **basic calculator** with + - * / and correct precedence. [Hashing / Simulation]

◆ *Concepts: Two stacks*

■ Hint: Apply higher-precedence operators before pushing a lower-precedence one.

151. Implement a **queue using two stacks**. [Hashing / Simulation]

◆ *Concepts: Amortised analysis*

■ Hint: Pour stack1 into stack2 only when stack2 is empty; then pop.

152. Solve the **Josephus problem** — every Kth person in a circle is removed. [Hashing / Simulation]

◆ *Concepts: Recursion*

■ Hint: josephus(n,k) = (josephus(n-1,k) + k) % n, with josephus(1,k) = 0.

153. Find the **minimum coins** to make change for an amount. [Hashing / Simulation]

◆ *Concepts: Greedy vs dynamic programming*

■ Hint: Greedy fails for coins like {1,3,4}; DP is the safe general answer.

154. Solve **Snakes and Ladders** — fewest dice rolls to reach 100. [Hashing / Simulation]

◆ *Concepts: BFS on a graph*

■ Hint: Each square is a node; edges are dice rolls plus snake/ladder jumps.

155. Find the **next lexicographic permutation** of a sequence. [Hashing / Simulation]

◆ *Concepts: In-place array manipulation*

■ Hint: Find the rightmost ascent, swap with its next larger element, reverse the suffix.

Hashing & Simulation Tips

- Prefix sum + hashing (148) is one of the most reusable patterns in array problems.
- The Josephus recurrence (152) is elegant — derive it from the position shift after one elimination.
- Coin change (153): confirm with DP; greedy is correct only for canonical coin systems.
- Snakes and Ladders (154) is a shortest-path problem — BFS gives the minimum roll count.

Reference — Complexity, Index & Study Plan

Time Complexity Quick Reference

The final column shows roughly the largest input size that finishes within one second, a useful sanity check when choosing an approach under given constraints.

Complexity	Name	Typical Algorithm	Feasible N (~1s)
`O(1)`	Constant	Hash lookup, arithmetic	Any
`O(log n)`	Logarithmic	Binary search, matrix power	10^18
`O(n)`	Linear	Single scan, two pointers	10^8
`O(n log n)`	Linearithmic	Merge sort, LIS (fast)	10^6
`O(n^2)`	Quadratic	Nested loops, simple DP	10^4
`O(n^3)`	Cubic	Triple loops, matrix chain	500
`O(2^n)`	Exponential	Power set, naive recursion	20-25
`O(n!)`	Factorial	All permutations	10-12

Problem Index by Module

Module	Range	Topic	Difficulty	Central Ideas
1	1-25	Basics	Easy	Loops, strings, arrays
2	26-50	Intermediate logic	Medium	Sieve, Kadane, Dutch flag
3	51-80	Patterns	Easy-Medium	Nested loops, boundaries
4	81-90	Sequences	Medium	Recurrences, formulas
5	91-105	Number theory	Medium	Divisors, totient, digits
6	106-120	Strings	Medium-Hard	Stacks, sliding window, DP
7	121-135	Arrays & matrix	Hard	LIS, BFS/DFS, mono-stack
8	136-145	Recursion	Hard	Backtracking, pruning
9	146-155	Hashing & sim	Medium-Hard	HashMap, BFS, two stacks

Final Practice Advice

- Attempt each problem honestly for 20-30 minutes before reading its hint.
- Re-solve a problem a different way to build flexibility — recursion versus iteration, for instance.
- Write your test cases before your solution; it forces you to understand the problem.
- Check the constraints: N up to 10^5 usually rules out O(n²) and demands O(n log n) or better.
- Lean on Java's standard toolkit — HashMap, ArrayDeque, PriorityQueue, TreeMap — with confidence.
- Review in batches of ten: which techniques keep reappearing? Those are your high-value patterns.